

AST-Aware Code Chunker Design for Codebases

Abstract Syntax Trees in RAG

Traditional character-based chunking breaks code mid-function or across class boundaries, destroying syntactic context. AST parsers parse source code into tree nodes representing classes, methods, and functions.

Tree-Sitter Integration

Tree-Sitter generates concrete syntax trees for Python, TypeScript, and C++. AST-based chunking splits source code along clean AST nodes, maintaining function signatures and docstrings intact.

Scope-Aware Context

Every AST chunk retains parent scope headers (e.g. enclosing class name and package imports) so LLMs receive fully qualified context.